Desenvolvimento de um Sistema Open-Source para Automação de Pesagem Rodoviária com LPR e Visão Computacional

Rafael B. Zanon[1], **Marcos V. Corino**[1]

[1]Curso de Análise e Desenvolvimento de Sistemas
Instituto Federal do Rio Grande do Sul – Veranópolis – Rio Grande do Sul – Brasil

rafael@email.com, marcos@email.com

Resumo. *Este trabalho apresenta o EasyScale, protótipo web open-source que automatiza a pesagem rodoviária de caminhões após o agendamento, integrando reconhecimento de placas com OpenCV e EasyOCR, validação do agendamento, controle de duas cancelas, conferência de posição, aceitação de pesos estáveis e cálculo do peso líquido. Desenvolvido com Python, FastAPI, PostgreSQL e React e avaliado em ambiente simulado, o protótipo passou em 1.177 de 1.178 testes automatizados; a única falha veio de um teste dependente da data. Dos 12 cenários operacionais, 11 foram verificados integralmente. O reconhecimento levou de 105 a 170 ms por imagem sintética em CPU, mas sua taxa de acerto não foi medida por falta de imagens reais.*

Abstract. *This paper presents EasyScale, an open-source web prototype that automates truck weighing after scheduling, integrating license plate recognition with OpenCV and EasyOCR, schedule validation, control of two gates, position checking, acceptance of stable weights, and net weight calculation. Built with Python, FastAPI, PostgreSQL, and React and evaluated in a simulated environment, the prototype passed 1,177 of 1,178 automated tests; the only failure came from a date-dependent test. Of 12 operational scenarios, 11 were fully verified. Recognition took 105 to 170 ms per synthetic image on a CPU, but its accuracy was not measured due to the lack of real images.*

1. Introdução

O transporte rodoviário é uma das principais formas de movimentação de cargas em empresas, indústrias, cooperativas e centros de distribuição. Nesses ambientes, a pesagem de caminhões controla a entrada e a saída de mercadorias: registra-se o peso do veículo antes e depois da carga ou descarga, apura-se o peso líquido e mantém-se a rastreabilidade das cargas. Em muitos casos, porém, o processo ainda depende de pessoas para conferir agendamentos, identificar o veículo, abrir cancelas, ler a balança e registrar as informações, o que o expõe a atrasos, erros de digitação e inconsistências nos registros.

Duas tecnologias permitem automatizar esse fluxo. O reconhecimento automático de placas veiculares (LPR, do inglês *License Plate Recognition*) identifica o veículo a partir de imagens [Du et al. 2013], e a comunicação serial no padrão RS232, comum em indicadores de pesagem e controladores, permite ler o peso e comandar dispositivos [Axelson 2007]. Os trabalhos sobre LPR, contudo, concentram-se na identificação, e as soluções que a integram à pesagem são, em geral, produtos comerciais de código fechado

(Seção 2.6). Faltam soluções abertas que reúnam identificação, agendamento e pesagem com regras explícitas para as situações de falha, nas quais um erro pode liberar o veículo errado ou registrar um peso incorreto.

Este trabalho desenvolveu o EasyScale, um protótipo de sistema web open-source que automatiza as etapas da pesagem posteriores ao agendamento, o qual permanece uma ação humana. O cenário operacional foi definido para o protótipo: um pátio com uma balança rodoviária, cancelas de entrada e de saída, câmeras de identificação e uma câmera superior, que atende operações de carga e descarga com duas pesagens e um caminhão por vez sobre a balança. Consideram-se apenas placas brasileiras, nos formatos antigo e Mercosul. Todos os equipamentos foram simulados: as imagens vêm de pastas de arquivos ou de uma câmera simulada que gera cenas sintéticas; balança, cancelas e posição são simuladores configuráveis; e a comunicação serial com a balança foi implementada apenas no nível de transporte, sem o protocolo de um indicador real. Ficaram fora do escopo a automação do agendamento, a orientação do motorista para posicionar o caminhão, a integração com sistemas de gestão, a emissão e a conferência de documentos fiscais e a cobrança.

A questão de pesquisa foi: *é viável integrar reconhecimento de placas, agendamento e comunicação com balança e cancelas em um fluxo de pesagem que, em caso de dúvida, não autoriza a entrada, não aciona cancelas e não registra peso, usando somente ferramentas de código aberto?* Dentro da delimitação acima, a resposta foi buscada na lógica e na integração do sistema, avaliadas em ambiente simulado. Ganhos de tempo ou redução de erros na operação real não foram medidos e são tratados como possibilidades futuras.

O objetivo geral foi desenvolver e avaliar, em ambiente simulado, um protótipo de sistema web open-source que integre reconhecimento automático de placas, agendamento, controle de cancelas e registro de pesagens em um fluxo de pesagem rodoviária com comportamento de falha segura. A Tabela 1 apresenta os objetivos específicos, a entrega que demonstra cada um e a forma de verificação.

Tabela 1. Matriz de acompanhamento dos objetivos específicos

Objetivo específico	Entrega que o demonstra	Forma de verificação
OE1. Especificar o sistema	Requisitos e regras de negócio com critérios de aceitação (Seção 3.2)	Correspondência com o problema e com a delimitação do escopo
OE2. Modelar a solução	Arquitetura, modelo de dados e máquina de estados (Seção 3.3)	Coerência entre modelos e implementação: teste de fronteiras entre módulos e restrições no banco
OE3. Implementar o reconhecimento de placas	Módulo de reconhecimento integrado à autorização (Seção 3.4)	Testes com imagens identificadas por gabarito
OE4. Integrar o processo de pesagem	Fluxo completo, com duas cancelas e duas pesagens (Seção 4)	Testes de entrada, das duas pesagens, de falhas e de encerramento
OE5. Avaliar o protótipo	Resultados e discussão (Seção 5)	Execução do protocolo de avaliação documentado (Seção 3.5)

A Seção 2 apresenta a fundamentação teórica e os trabalhos relacionados; a Seção 3, a metodologia; a Seção 4, o produto; a Seção 5, os resultados e sua discussão; e a Seção 6, as considerações finais.

2. Fundamentação Teórica e Trabalhos Relacionados

2.1. Reconhecimento automático de placas

Sistemas de ALPR (*Automatic License Plate Recognition*) dividem-se, em geral, em aquisição da imagem, localização da placa, segmentação e reconhecimento dos caracteres [Du et al. 2013]. O desempenho depende da robustez de cada etapa diante de iluminação, ângulo, distância, sujeira e reflexos, e ambientes externos acrescentam variações ao longo do dia [Chang et al. 2004]. Abordagens recentes usam redes neurais treinadas para detectar o veículo, a placa e os caracteres, como o detector YOLO [Laroca et al. 2018]; elas exigem conjuntos de imagens anotadas, e seu desempenho cai quando avaliadas em bases diferentes daquelas em que foram treinadas [Laroca et al. 2022]. Em bases brasileiras com cenários reais, sistemas comerciais testados ficaram abaixo de 70% de reconhecimento [Laroca et al. 2018, Laroca et al. 2022], o que mostra a importância de avaliar o reconhecimento com imagens do ambiente de uso.

No Brasil, convivem o formato antigo (`AAA9999`) e o padrão Mercosul (`AAA9A99`), e a regulamentação define a equivalência entre eles pela troca do segundo algarismo da placa antiga por uma letra [CONTRAN 2022]. Essa convivência cria uma dificuldade específica: na quinta posição, tanto um algarismo quanto uma letra são válidos, de modo que uma confusão do OCR nessa posição pode produzir outra placa válida.

2.2. Processamento de imagens e OCR

O pré-processamento de imagens envolve transformações como conversão para tons de cinza, filtragem e detecção de bordas, seguidas da segmentação de regiões [Gonzalez and Woods 2008]. O detector de Canny [Canny 1986], formulado a partir dos critérios de boa detecção, boa localização e resposta única a cada borda, é usado com frequência antes da extração de contornos, cujos retângulos envolventes podem ser filtrados por área e proporção para encontrar regiões candidatas a placa; a biblioteca OpenCV oferece essas operações [OpenCV 2025]. Essa localização dispensa treinamento, mas é mais sensível às condições da imagem do que os detectores treinados citados acima.

O OCR (*Optical Character Recognition*) converte o texto de uma região da imagem em caracteres e, em geral, atribui a cada leitura um índice de confiança. Esse índice permite descartar leituras duvidosas, mas não equivale, necessariamente, à probabilidade de a leitura estar correta; por isso, o limiar de aceitação precisa ser ajustado com imagens do ambiente de uso. Em placas, os erros mais recorrentes são trocas entre letras e algarismos de forma parecida, como B e 8, G e 6 ou I e 1. Quando o formato da placa determina se cada posição tem letra ou algarismo, essas trocas podem ser desfeitas por regras aplicadas após o reconhecimento [Laroca et al. 2021].

O EasyOCR é uma biblioteca de OCR de código aberto, sob licença Apache 2.0, baseada em aprendizado profundo, com modelos pré-treinados que funcionam em CPU [Jaided AI 2025], já combinada com o OpenCV em sistemas de reconhecimento de placas em Python [Rao et al. 2025]. Isso justificou sua adoção, prevista desde a proposta: ele se integra ao backend em Python e dispensa um conjunto de treinamento, que o trabalho não tinha. Em contrapartida, detectores treinados para placas tendem a ser mais robustos [Laroca et al. 2018], e alternativas de OCR não foram comparadas.

2.3. Comunicação serial com indicadores de pesagem

Na comunicação serial, os bits são transmitidos em sequência por um canal, e as duas pontas precisam combinar porta, taxa de transmissão, paridade e quantidade de bits de dados e de parada [Axelson 2007]. Como a resposta do equipamento pode não chegar, cada leitura precisa de um tempo-limite. Indicadores de pesagem transmitem o peso em quadros de texto, de forma contínua ou sob pedido; a posição do peso, os delimitadores do quadro, a unidade e as marcas de estabilidade ou de erro variam conforme o fabricante e o modelo, de modo que interpretar os dados exige o protocolo do equipamento.

2.4. Pesagem rodoviária e agendamento

Na pesagem rodoviária, a tara é o peso do veículo vazio, o peso bruto é o do veículo carregado e o peso líquido é o da carga. Quando o caminhão passa duas vezes pela balança, antes e depois da operação, o peso líquido é a diferença entre as pesagens: na carga, a final menos a inicial; na descarga, a inicial menos a final. Como a indicação oscila enquanto o veículo se acomoda sobre a plataforma, o peso só deve ser registrado depois de estabilizado. Para organizar as chegadas, terminais logísticos usam sistemas de agendamento em que cada caminhão reserva uma janela de tempo e cada janela tem uma quota máxima de agendamentos [Gracia et al. 2025]; o protótipo adota a mesma ideia, com faixas de horário de capacidade definida.

2.5. Sistemas web

Uma API REST expõe operações HTTP padronizadas entre cliente e servidor. Para atualizar a interface sem recarregar a página, o mecanismo SSE (*Server-Sent Events*) mantém aberta uma conexão HTTP pela qual o servidor envia eventos [WHATWG 2026], o que basta quando só o servidor precisa avisar mudanças. Sessões guardadas no servidor e identificadas por *cookie* podem ser encerradas e expiradas no próprio servidor. Um monólito modular é implantado como um único processo, mas dividido em módulos com fronteiras explícitas, o que preserva a separação de responsabilidades sem a complexidade de serviços distribuídos; à sua frente, um *proxy* reverso pode encerrar o HTTPS e entregar os arquivos estáticos da interface.

2.6. Trabalhos relacionados

Gomes [Gomes 2016] automatizou a balança rodoviária de uma fábrica: a identificação usa etiquetas RFID, um CLP lê o peso do indicador pela porta serial, e um sistema supervisório libera duas cancelas, com semáforos e sensores de presença. O trabalho não relata medições e registra problemas de campo, como a leitura de etiquetas alfanuméricas e a perda de detecção com bitrens. Valle Júnior e Veronez Júnior [Valle Júnior and Veronez Júnior 2018] modernizaram as balanças de uma siderúrgica, substituindo a interação entre motorista e sistema por RFID, e relatam redução do tempo de pesagem. Nas duas soluções, a identificação depende de etiquetas distribuídas aos motoristas.

Agrawal e Joshi [Agrawal and Joshi 2022] tratam do reconhecimento de placas de caminhões para automatizar balanças na Índia, com uma base própria de imagens coletadas em balanças, detecção por YOLOv7 e reconhecimento por PARSeq, e relatam 95,82% de acerto; o trabalho cobre apenas o reconhecimento. Rao et al. [Rao et al. 2025] estudam

o reconhecimento de placas em Python com OpenCV e EasyOCR, combinação próxima à deste trabalho, sem integração a um processo operacional. Entre as soluções comerciais, o SmarTower [Avery Weigh-Tronix 2026] automatiza balanças sem operador com identificação por RFID e reconhecimento de placas opcional, sensores de posição e cancelas opcionais; e o sistema da ESSMANN [ESSMANN 2026] usa reconhecimento de placas em balanças de entrada e saída, processa automaticamente pesagens pré-cadastradas e detecta a posição do veículo. Ambos são produtos comerciais, sem código disponível.

A Tabela 2 resume a comparação. Os trabalhos acadêmicos tratam da identificação ou da automação da balança, e as soluções que integram as duas são comerciais. A contribuição do EasyScale está em reunir, em código aberto, reconhecimento de placas, agendamento por faixas de horário, integração com a pesagem e controle de acesso com confirmação das cancelas, sob regras explícitas de falha segura.

Tabela 2. Comparação entre os trabalhos relacionados (Seção 2.6) e o protótipo; ND: não documentado na fonte

Solução	Reconhecimento de placas	Agendamento	Integração com pesagem	Controle de acesso	Código disponível
Gomes (2016)	Não (RFID; câmeras só registram)	Não	Sim (CLP e indicador serial)	Sim (duas cancelas e semáforos)	Não
Valle Júnior e Veronez Júnior (2018)	ND (identificação por RFID)	ND	Sim	ND	ND
Agrawal e Joshi (2022)	Sim (YOLOv7 e PARSeq)	ND	ND	ND	ND
Rao et al. (2025)	Sim (OpenCV e EasyOCR)	ND	ND	ND	ND
SmarTower (Avery Weigh-Tronix)	Opcional (identificação por RFID)	ND	Sim	Sim (cancelas opcionais)	Não (comercial)
ESSMANN	Sim	Pesagens pré-cadastradas	Sim	Sim	Não (comercial)
EasyScale	Sim (OpenCV e EasyOCR)	Sim (faixas com capacidade e totem)	Sim (simulada; serial no nível de transporte)	Sim (cancelas com confirmação e perfis)	Sim (licença MIT)

3. Metodologia

3.1. Caracterização e processo de desenvolvimento

Trata-se de pesquisa aplicada, de natureza tecnológica, com desenvolvimento e avaliação de um protótipo. O desenvolvimento foi orientado por especificação: cada funcionalidade passou por requisitos, especificação (comportamento normal e em falha e critérios de aceitação verificáveis), análise de impacto, plano, implementação, testes e revisão, em 29 especificações, e cada teste cita o critério de aceitação que verifica. As decisões de arquitetura foram registradas em oito ADRs (*Architecture Decision Records*), com contexto, opções e consequências [Nygard 2011]. Adotou-se como princípio a falha segura (*fail-safe*): diante de qualquer dúvida, o sistema não autoriza, não aciona cancelas e não registra peso, e encaminha a situação ao operador.

Em relação à proposta, a interface passou a ser escrita em TypeScript e React, em vez de HTML e JavaScript; o banco foi definido como PostgreSQL; o fluxo passou a usar duas cancelas e uma câmera superior para conferir a posição do caminhão; e foram

acrescentados o autoatendimento do motorista, a tela de diagnóstico do reconhecimento e o perfil de auditor.

3.2. Contexto e levantamento de requisitos

Os requisitos vieram de duas fontes: a proposta deste trabalho, da qual foram extraídos os requisitos de cada seção, e decisões do autor registradas na documentação do projeto. Não houve entrevistas nem participação de uma empresa: o cenário foi definido para o protótipo, e os pontos ambíguos foram convertidos em perguntas, respondidas pelo autor, sobretudo em 29 de outubro e 4 de novembro de 2026. Também foram consultadas a regulamentação das placas [CONTRAN 2022], para a conversão entre os formatos, a instrução normativa que cria o CNPJ alfanumérico [Receita Federal do Brasil 2024], para o cadastro de transportadoras, e as recomendações da OWASP para armazenar senhas [OWASP Foundation 2026]. Cada requisito tem identificador estável e origem indicada. Não houve priorização formal: os requisitos da proposta foram tratados como obrigatórios, e a implementação seguiu as dependências técnicas, do domínio até a interface e a integração. O levantamento resultou em 34 requisitos funcionais (RF), um deles retirado do escopo, 14 não funcionais (RNF) e 24 regras de negócio (RN). A Tabela 3 apresenta os principais, com os critérios de aceitação.

Tabela 3. Principais requisitos e regras de negócio, com critérios de aceitação

Identificador	Requisito	Critério de aceitação
RF-009, RN-002	Validar o agendamento da placa reconhecida	Autorizar só com placa em formato válido e confiança igual ou superior ao limiar, veículo e motorista cadastrados e ativos, nenhuma operação aberta do veículo e um único agendamento válido
RN-014	Respeitar a validade do agendamento	Não autorizar antes da faixa de horário nem em outro dia; autorizar depois da faixa, no mesmo dia
RN-004	Aceitar somente pesos válidos	Aceitar a última leitura de uma série estável (quantidade mínima, variação máxima, janela de tempo e faixa plausível); nunca ajustar leitura rejeitada
RF-014, RN-012	Registrar as duas pesagens da operação	Ler o peso só com o caminhão em posição; liberar a saída só após a pesagem aceita; encerrar só após a pesagem final
RF-016, RN-006	Calcular o peso líquido	Carga: final $-$ inicial; descarga: inicial $-$ final; entre -20 e $+20$ kg, sem movimentação; abaixo disso, intervenção do administrador
RN-007	Comandar as cancelas com segurança	Comandar só se o estado permitir; sem confirmação do sensor, estado desconhecido e parada para intervenção; no máximo duas tentativas
RN-019	Atender um caminhão por vez	Abrir a entrada só com a balança livre e reservada para a operação
RF-033, RN-021	Permitir o autoatendimento do motorista	Criar o agendamento só se placa e CPF corresponderem aos cadastros, sem revelar qual dado falhou

Entre os requisitos não funcionais, têm verificação objetiva a separação em módulos (RNF-001), por um teste que falha se uma camada importar outra indevidamente, o modo simulado como padrão (RNF-003), o uso do PostgreSQL (RNF-004), os tempos-limite da comunicação serial (RNF-005), a falha segura (RNF-009) e o registro de eventos para auditoria (RNF-010). São verificadas em parte a ausência de placa, CPF

e nota nos registros de *log* (RNF-013), sem avaliação formal de adequação à Lei Geral de Proteção de Dados (LGPD, Lei nº 13.709/2018), e a avaliação reprodutível do reconhecimento (RNF-012), não executada com imagens reais. A navegação simples (RNF-002) e o não bloqueio da API durante o reconhecimento (RNF-011) não têm verificação objetiva.

3.3. Modelagem

A Figura 1 apresenta a arquitetura, um monólito modular cujas fronteiras entre módulos são verificadas por teste automatizado. O módulo `dominio` reúne regras puras (normalização e seleção de placa, autorização, aceitação do peso e máquina de estados), sem acesso a banco, câmera ou porta serial; `orquestracao` conduz o fluxo e reserva a balança; `lpr` contém as fontes de imagem, o reconhecimento e a fonte de posição; `dispositivos` implementa balança e cancelas; `persistencia` concentra o acesso ao banco; e `api` expõe as rotas, a sessão e as permissões. O servidor Caddy entrega a interface com HTTPS e repassa as requisições à API.

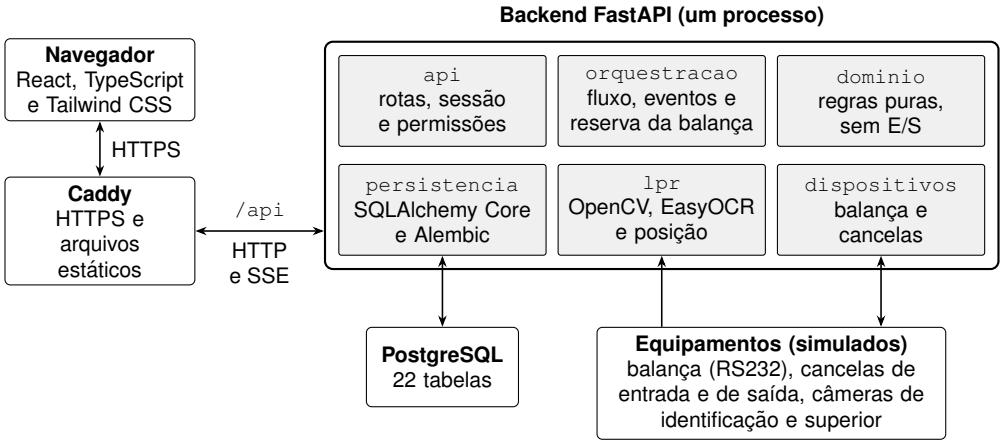


Figura 1. Arquitetura do protótipo: monólito modular com fronteiras entre módulos verificadas por teste automatizado

A Figura 2 apresenta o modelo de dados, que tem 22 tabelas e 35 chaves estrangeiras; a figura omite as tabelas de acesso e auditoria (perfil, permissões, usuário, sessão, tentativas de acesso e registro de auditoria) e as referências secundárias, como as que apontam o usuário responsável. Algumas regras de negócio são garantidas pelo próprio banco: índices únicos parciais impedem mais de uma pesagem não cancelada por agendamento, mais de uma operação aberta por veículo, mais de uma operação com a balança reservada e mais de um agendamento aberto por veículo; uma restrição de exclusão impede faixas de horário ativas sobrepostas; e uma verificação confere a fórmula do peso líquido. As imagens que fundamentam decisões são gravadas no banco, na mesma transação da decisão.

A Figura 3 apresenta a máquina de estados da operação, com 20 situações. O fluxo normal percorre as duas passagens pela balança. A situação `AGUARDANDO_INTERVENCAO` é uma parada segura, sem comandos, que guarda a situação anterior e um de seis motivos: falha de cancela, cancela sem confirmação, posição perdida, tempo esgotado, peso líquido negativo e reinício inconclusivo. Ela só termina por ação humana, com responsável e motivo, que retoma a etapa interrompida, confirma ma-

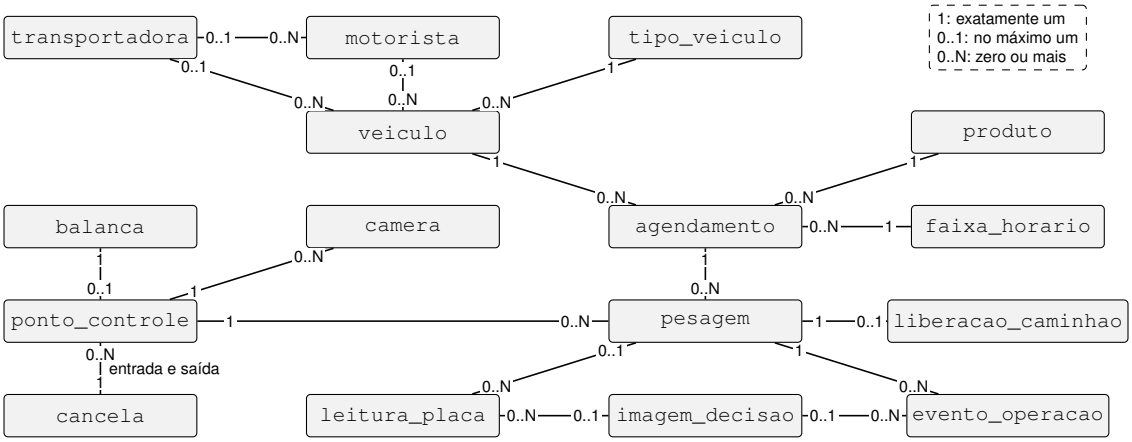


Figura 2. Modelo de dados: tabelas centrais e chaves estrangeiras (omitidas as tabelas de acesso e auditoria e as referências secundárias, como de *evento_operacao* a *leitura_placa*)

nualmente uma condição ou aceita o peso final. Transições inválidas são rejeitadas e registradas, e ações com versão desatualizada da operação são recusadas.

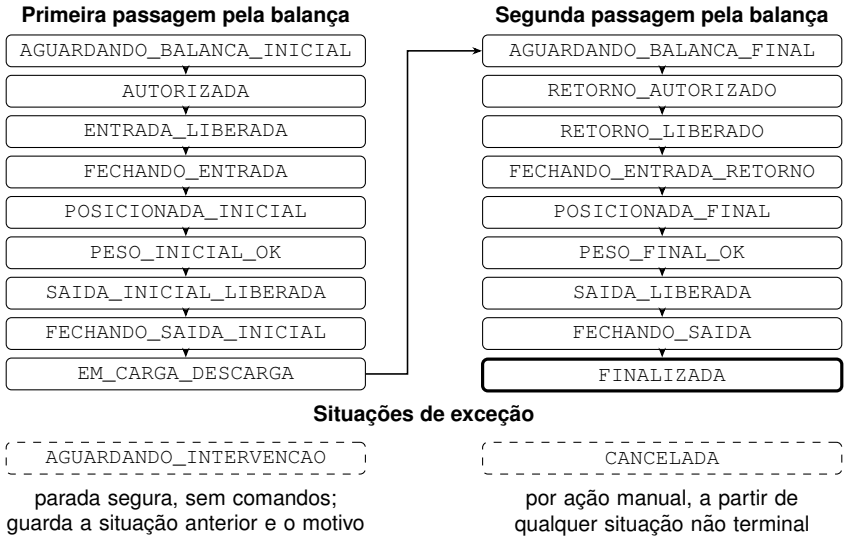


Figura 3. Situações da máquina de estados da operação: fluxo normal nas duas passagens pela balança e situações de exceção

3.4. Decisões de implementação

O PostgreSQL [PostgreSQL 2026] foi escolhido pelas transações e pelas restrições declarativas que sustentam as regras acima; o acesso usa SQLAlchemy Core, com SQL explícito, e o esquema evolui por quatro migrações Alembic. O backend usa FastAPI [Ramírez 2025]; a interface, React [React 2026] e TypeScript [TypeScript 2026]; e a execução, Docker Compose [Docker 2026]. A API tem 67 rotas no modo simulado (62 quando algum dispositivo é real): cadastros (20), agendamentos (10), operação (11), histórico (2), diagnóstico (7), autenticação e usuários (9), autoatendimento (3) e, só no modo simulado, simuladores (5). Ela valida as entradas, controla sessão e permissões e envia os eventos da operação por SSE; nenhuma decisão de segurança fica na interface.

O reconhecimento converte a imagem para tons de cinza, detecta bordas com o algoritmo de Canny e mantém, entre os contornos, os retângulos com área e proporção compatíveis com uma placa. Essa localização foi preferida ao detector de texto do próprio EasyOCR aplicado à imagem inteira por seguir a proposta e produzir as regiões que a tela de diagnóstico exibe; usar os dois dobraria o custo e aumentaria a chance de ambiguidade. Cada região é lida pelo EasyOCR [Jaided AI 2025] sem restringir o alfabeto, para que um caractere estranho não vire um "parecido" que forme uma placa válida, porém errada. A normalização aceita só os formatos antigo e Mercosul e, nas leituras de câmera, corrige conforme a posição oito confusões entre letra e algarismo (O/0, I/1, B/8, S/5, Z/2, G/6, A/4 e T/7), registrando cada correção; a quinta posição nunca é corrigida, pois aceita algarismo no formato antigo e letra no Mercosul. A seleção é conservadora: sem candidato válido ou com duas placas diferentes, não há placa; a confiança de uma placa é a menor entre as suas leituras; e, abaixo do limiar (0,800 na demonstração), a leitura é rejeitada. Os parâmetros de localização usados nos testes (limites do Canny de 50 e 150, área mínima de 2.000 px², proporção entre 2 e 6 e até cinco regiões) não foram calibrados.

Na balança serial, implementada com PySerial [pySerial 2025], protocolo, porta, taxa de transmissão, paridade, bits de dados e de parada, tempo-limite e intervalo entre tentativas vêm de variáveis de ambiente, sem valores padrão; sem uma delas, o sistema não parte. A cada leitura, o sistema descarta os bytes pendentes, para nunca usar um peso antigo, envia o pedido (quando o protocolo exige), acumula bytes até o tempo-limite e interpreta o primeiro quadro completo pelo protocolo do modelo de indicador; quadro ilegível, tempo esgotado e erro de porta resultam em quadro corrompido, ausência de resposta (com uma segunda tentativa) e desconexão. O registro de protocolos está vazio, com um roteiro documentado para integrar um indicador a partir do manual e de quadros reais; a cancela serial não foi implementada.

O peso aceito é a última leitura de uma série estável, sem média nem arredondamento. Na demonstração, a balança é lida a cada 500 ms, e a série precisa de três leituras com variação de até 10 kg em uma janela de 3 s, todas entre 100 e 80.000 kg; a tolerância de movimentação é de 20 kg. Esses valores também não foram calibrados.

Os simuladores substituem os equipamentos, com cenários trocados por rotas da API disponíveis só no modo simulado. A balança devolve um peso configurado, fixo ou, no cenário instável, alternando acima e abaixo dele, e reproduz ausência de resposta, quadro corrompido e desconexão. As cancelas reproduzem confirmação, falha, cancela que não se move e sensor sem resposta, com no máximo duas tentativas, e seu estado vem sempre do sensor, nunca do comando. A posição pode ser confirmada, ausente ou inconclusiva. Na câmera simulada, a placa é desenhada sobre uma cena sintética, e o reconhecedor simulado devolve textos e confianças enfileirados previamente, sem executar o OCR, que só roda na avaliação do reconhecimento.

O acesso é controlado por três perfis, com permissões por ação: administrador (14 permissões), operador (8) e auditor (3, só de consulta). As senhas são guardadas com `scrypt`, a sessão fica no banco e o *cookie* é restrito e expira por inatividade. Intervenções, liberações manuais, cancelamentos e exclusões de imagem registram responsável e motivo.

3.5. Protocolo de avaliação

A avaliação foi organizada em três partes, definidas a partir dos requisitos e das especificações, e executada em 8 de novembro de 2026 sobre a versão `f4a97a0` do repositório, em um computador com AMD Ryzen 7 5700X (8 núcleos e 16 *threads*), 62 GiB de memória e sem GPU, com Arch Linux (*kernel* 7.2.8), Docker 29.8, Docker Compose 5.7 e PostgreSQL 18.6 em contêiner. O backend usou Python 3.14, FastAPI 0.162, SQLAlchemy 2.1, Alembic 1.20 e PySerial 3.5; o reconhecimento, OpenCV 4.14, EasyOCR 1.7.2 e PyTorch 2.14 em CPU; a interface, TypeScript 6.0, React 19.3, Tailwind CSS 4.3 e Vite 8.3; e os testes, pytest 9.1, Vitest 5.0 e Testing Library. Por uma limitação da rede do Docker na máquina, o PostgreSQL dos testes rodou em contêiner na rede do hospedeiro, com a mesma imagem e os mesmos testes.

1. **Testes automatizados** em três níveis: unitários (regras do domínio, sem banco, câmera ou porta serial), de integração (API com PostgreSQL real) e de ponta a ponta (fluxo completo pela API), além dos testes da interface. Registram-se, a partir das saídas do pytest e do Vitest, quantidade, aprovação, tempo e cobertura de código.
2. **Cenários operacionais**: doze cenários definidos a partir dos requisitos e da proposta, cada um com condição inicial, resultado esperado, resultado obtido e situação.
3. **Reconhecimento de placas**: acervo de imagens reais, guardado fora do repositório por conter dados pessoais, com gabarito da placa esperada em cada imagem (vazia se ilegível) e subpastas por condição, como diurno, noturno e inclinadas. Cada imagem é classificada como acerto, aceitação incorreta (falso positivo de autorização, o erro mais grave), rejeição segura (falso negativo, que vai ao operador) ou rejeição correta, e um relatório sem placas registra as taxas, no total e por condição, e a latência média e máxima. Não há meta numérica, e a infraestrutura não calcula o acerto por caractere. Sem acervo disponível, essa parte não foi executada; mediu-se só o tempo de processamento em duas imagens sintéticas, com cinco execuções de aquecimento e 50 medidas cada.

4. Produto

O EasyScale conduz cada operação pela máquina de estados da Figura 3. Quando um caminhão agendado chega, a placa é reconhecida e a autorização é decidida pelas regras da Tabela 3. Autorizada, a operação aguarda a balança ficar livre, abre a entrada, espera a câmera superior confirmar a posição, fecha a entrada, lê o peso até obter uma série estável, registra a pesagem inicial e abre a saída. Depois da carga ou descarga, a placa é lida de novo, e o caminhão repete a passagem para a pesagem final; a operação termina quando a saída fecha com a balança vazia. Cada comando de cancela é confirmado pelo sensor, e cada passo gera um evento no histórico. O código-fonte está em repositório público[1], com licença MIT, documentação de instalação e execução, requisitos, especificações e ADRs.

A leitura da balança chega à orquestração como peso e instante, é avaliada pela regra de estabilidade e é exposta à interface pela API e por SSE. A Figura 4 mostra mensagens de uma execução real, com a balança simulada em 15.230 kg; como o peso lido é enviado com intervalo mínimo de um segundo e a balança é lida a cada 500 ms,

[1] `https://github.com/rafaelbzanon/easyscale`

nem todas as leituras aparecem no canal. A primeira avaliação ainda rejeita o peso, por falta de leituras estáveis suficientes, e a seguinte o aceita, o que gera a transição para `PESO_INICIAL_OK`, o comando de abertura da saída e sua confirmação.

```
event: peso_lido
data: {"peso_kg":"15230.000"}
event: balanca
data: {"modo":"SIMULADO","comunicacao":"OK","motivo_falha":null,
       "ultima_leitura":{"peso_kg":"15230.000"},
       "avaliacao_peso":{"aceito":false,
                         "motivo_rejeicao":"LEITURAS_INSUFICIENTES"}}
event: balanca
data: {...,"ultima_leitura":{"peso_kg":"15230.000"},
       "avaliacao_peso":{"aceito":true,"motivo_rejeicao":null}}

 9|EVENTO_REGISTRADO|POSICAO_CONFIRMADA|POSICIONADA_INICIAL
10|TRANSICAO|PESO_ACEITO|POSICIONADA_INICIAL|PESO_INICIAL_OK|15230.000
11|COMANDO_CANCELA|ABRIR_SAIDA|PESO_INICIAL_OK| ... SAIDA|ABRIR|CONFIRMADO|1
12|TRANSICAO|CANCELA_CONFIRMADA|PESO_INICIAL_OK|SAIDA_INICIAL_LIBERADA
```

Figura 4. Mensagens SSE da balança simulada durante a pesagem inicial (campos de horário omitidos) e registros correspondentes na tabela *evento_operacao*: sequência, tipo, evento, situação de origem, situação de destino e peso ou dados do comando

A tela de operação (Figura 5) mostra em tempo real a placa reconhecida, a decisão, o estado das cancelas, a etapa e o peso lido, marcado como "não é peso aceito" até ser aceito. No exemplo, uma operação de carga com pesagem inicial de 15.000 kg e final de 45.000 kg resulta em 30.000 kg de peso líquido; os pesos medidos e o líquido são gravados como lidos, nunca ajustados. O cabeçalho indica sempre o modo (simulado ou real) e, se o servidor deixar de responder, a interface bloqueia as ações.

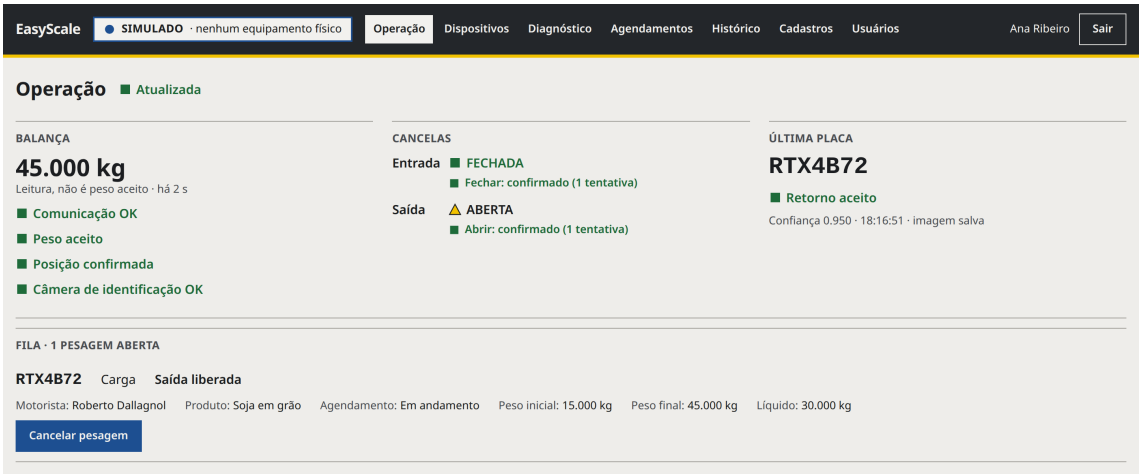


Figura 5. Tela de operação após a pesagem final de uma carga: peso final aceito e peso líquido calculado (45.000 − 15.000 = 30.000 kg), em modo simulado

A tela de diagnóstico (Figura 6) mostra, para cada reconhecimento, a imagem com as regiões analisadas, o texto lido, a placa normalizada, o formato, as correções, a confiança e a decisão, inclusive das leituras não autorizadas. A Figura 7 mostra a resposta a uma falha: sem confirmação do sensor depois de duas tentativas, a cancela de entrada passa a estado desconhecido, a operação para em intervenção, e a nova tentativa exige um motivo.



Figura 6. Tela de diagnóstico: leituras recentes, uma delas não autorizada por falta de agendamento, e detalhe de uma autorização, com a imagem sintética da câmera, a região da placa, o texto lido, a placa normalizada, o formato, as correções e a confiança



Figura 7. Resposta a uma falha: cancela de entrada sem confirmação (estado desconhecido), operação aguardando intervenção e pedido de nova tentativa com motivo

A Tabela 4 relaciona as funcionalidades aos requisitos que atendem e à evidência apresentada neste artigo; os cenários T1 a T12 estão na Tabela 6.

5. Resultados e Discussão

5.1. Testes automatizados

A Tabela 5 resume a execução dos testes. Dos 1.002 testes do backend, 1.001 passaram. O que falhou verifica os filtros do histórico: cria pesagens com a data do dia e espera que nenhuma apareça no filtro de 8 de novembro de 2026, o que deixou de valer quando a execução coincidiu com essa data. É um defeito do teste, que depende do relógio, e não do sistema; a correção (fixar a data) fica pendente. A cobertura de linhas, considerando todas

Tabela 4. Relação entre funcionalidades, requisitos e evidências (o RF-012 foi retirado do escopo)

Funcionalidade	Requisitos	Evidência
Cadastros	RF-001 a RF-003, RF-025, RF-026	Testes da API e da tela
Agendamento e autoatendimento	RF-004, RF-005, RF-028, RF-029, RF-033; RN-001, RN-013 a RN-015, RN-021, RN-023, RN-024	Testes da API e das telas; T3, T4
Identificação e autorização	RF-006 a RF-010, RF-034; RN-002, RN-003, RN-014, RN-017, RN-018	Figura 6; T1, T2, T5, T6
Máquina de estados da operação	RF-015; RN-008 a RN-010; RNF-009	Figura 3; T6, T9
Cancelas, posição e balança	RF-011, RF-013, RF-017, RF-027, RF-030; RN-004, RN-005, RN-007, RN-012, RN-019	Figuras 4 e 7; T7, T8, T12
Pesagens e peso líquido	RF-014, RF-016; RN-006	Figuras 4 e 5; T9, T10
Tela operacional em tempo real	RF-021, RF-023	Figura 5
Histórico e auditoria	RF-018 a RF-020; RNF-010	T11
Diagnóstico e intervenção	RF-022, RF-024; RN-011, RN-016, RN-020, RN-022	Figuras 6 e 7; T5, T12
Usuários e acesso	RF-031, RF-032; RNF-013, RNF-014	Testes de permissão

as suítes, foi de 100% no domínio, 95,5% na orquestração, 97,9% no reconhecimento, 98,9% nos dispositivos, 98,5% na persistência e 92,4% na API.

Tabela 5. Execução dos testes automatizados em 8 de novembro de 2026

Conjunto	Testes	Aprovados	Falhos	Tempo (s)
Backend: unitários	861	861	0	8,8
Backend: integração (API com PostgreSQL)	104	103	1	38,4
Backend: ponta a ponta	18	18	0	18,3
Backend: avaliação do reconhecimento (testes auxiliares, sem OCR)	19	19	0	0,1
Backend: total (cobertura de linhas e ramos: 94,9%)	**1.002**	**1.001**	**1**	60,2
Frontend	176	176	0	5,1

5.2. Cenários operacionais

A Tabela 6 apresenta os doze cenários, verificados por testes de ponta a ponta, de integração e unitários; onze foram verificados integralmente. O cenário T3 é parcial: o agendamento vencido tem teste, mas o cancelado não; o código só considera agendamentos com situação `AGENDADO`, o que impede a autorização, mas isso foi verificado apenas por leitura. No T11, a única falha foi a do teste dependente da data. Os onze grupos de casos de uso da interface também foram percorridos no navegador, em 7 de novembro de 2026, sem falhas funcionais.

5.3. Reconhecimento de placas

Sem acervo de imagens reais, a taxa de acerto e os resultados por condição não puderam ser medidos. O reconhecimento completo levou, em CPU, 105,0 ms em média (mediana de 103,9 ms e percentil 95 de 126,8 ms) em uma imagem sintética de 640×480 pixels com duas regiões candidatas e 169,9 ms (mediana de 165,9 ms e percentil 95 de 215,0 ms) na

Tabela 6. Cenários operacionais: condição inicial, resultado esperado, resultado obtido e situação

Teste	Condição inicial	Resultado esperado	Obtido	Situação
T1. Agendamento válido	Veículo e motorista ativos; agendamento de hoje, na faixa; confiança $\geq$ 0,800	Autorizada; pesagem criada; entrada aberta	Conforme	Aprovado
T2. Sem agendamento	Placa cadastrada sem agendamento; placa desconhecida	Não autorizada, com motivo; imagem guardada; sem pesagem nem comando	Conforme	Aprovado
T3. Agendamento vencido ou cancelado	Agendamento de outro dia; agendamento que não está `AGENDADO`	Não autorizada; vencido marcado como expirado	Vencido: conforme; cancelado: sem teste	Parcial
T4. Antes da faixa	Agendamento de hoje; chegada antes do início da faixa	Não autorizada	Conforme	Aprovado
T5. Falha ou baixa confiança	Sem texto; formato inválido; duas placas; confiança abaixo do limiar; falha da câmera	Não autorizada, com motivo; liberação só pelo administrador	Conforme	Aprovado
T6. Reconhecimento repetido	Mesma placa lida de novo com a operação aberta	Nenhuma operação nova; decisão registrada uma vez	Conforme	Aprovado
T7. Peso válido e inválido	Estável em 15.000 kg; instável ($\pm$50 kg); 50 kg, abaixo do mínimo; quadro corrompido	Só o estável é aceito (última leitura); os demais não gravam peso	Conforme	Aprovado
T8. Comunicação interrompida	Balança sem resposta ou desconectada, com o caminhão em posição	Sem peso; falha sinalizada; intervenção após o tempo-limite	Conforme	Aprovado
T9. Duas pesagens	Fluxo completo de carga	Pesagens de 15.000 e 45.000 kg registradas; oito comandos de cancela	Conforme	Aprovado
T10. Carga e descarga	Carga 15.000 $\rightarrow$ 45.000; descarga 40.000 $\rightarrow$ 15.000; 15.000 $\rightarrow$ 15.012; 30.000 $\rightarrow$ 20.000 (kg)	30.000; 25.000; 12, sem movimentação; $-$10.000, intervenção do administrador	Conforme	Aprovado
T11. Encerramento e histórico	Operação percorrida até o fim	Situação finalizada; cancelas fechadas; histórico com pesos e eventos; consulta pelo auditor	Conforme	Aprovado
T12. Cancela sem confirmação	Cancela com falha ou que não se move; sensor sem resposta	Intervenção; estado desconhecido; nova tentativa com responsável e motivo	Conforme	Aprovado

cena de 640$\times$360 pixels do simulador, com três regiões; o tempo cresce com o número de regiões, pois o OCR roda uma vez por região.

As duas imagens mostraram um acerto e uma falha que ilustra a diferença entre acertar caracteres e acertar a placa. Na cena do simulador, as três regiões leram a placa corretamente, aceita com a menor das confianças, 0,907. Na outra imagem, o OCR leu `ABC1023` no lugar de `ABC1D23`, com confiança de 0,956, e a seleção aceitou o texto como placa válida no formato antigo: seis dos sete caracteres estavam certos, mas a placa

estava errada, o que o protocolo classifica como aceitação incorreta. A troca ocorreu na quinta posição, que não é corrigida por aceitar algarismo e letra; em operação, essa leitura só autorizaria a entrada se correspondesse a outro veículo cadastrado com agendamento válido, mas mostra um risco real de falso positivo.

5.4. Discussão

Os resultados respondem à questão de pesquisa de forma parcial. A integração entre identificação, agendamento, cancelas e balança, com falha segura, mostrou-se viável: nos cenários de falha, a operação para sem autorizar, comandar ou registrar peso indevidamente, e as regras centrais têm testes e restrições no banco. Contribuíram para isso as regras do domínio sem entrada e saída, testadas com 100% de cobertura de linhas; a máquina de estados explícita, com parada segura, que tornou as falhas previsíveis; e os simuladores, que reproduzem falhas difíceis de provocar com equipamentos reais. As falhas encontradas no desenvolvimento vieram de lógica e concorrência, como um cancelamento seguido de um comando de cancela já agendado e ciclos de orquestração parados com a API ainda respondendo; ambas foram corrigidas, com testes de regressão.

A viabilidade do reconhecimento em condições reais, porém, não foi demonstrada. Imagens reais derrubam o desempenho de sistemas de reconhecimento [Laroca et al. 2018, Laroca et al. 2022], e a falha na quinta posição mostra que a seleção conservadora reduz, mas não elimina, o risco de falso positivo. Diferentemente das soluções com RFID [Gomes 2016, Valle Júnior and Veronez Júnior 2018], que evitam erros de leitura óptica ao custo de distribuir etiquetas, o EasyScale depende da qualidade das imagens; e, diferentemente das soluções comerciais [Avery Weigh-Tronix 2026, ESSMANN 2026], tem código aberto e regras explícitas, mas não foi validado em campo.

6. Considerações Finais

Este trabalho tratou da dependência de intervenção humana na pesagem rodoviária e desenvolveu o EasyScale, protótipo open-source que integra reconhecimento de placas, agendamento, controle de cancelas e registro de pesagens com falha segura, avaliado em ambiente simulado. Foram atingidos a especificação do sistema (OE1), com 34 requisitos funcionais, 14 não funcionais e 24 regras de negócio detalhados em 29 especificações, e a modelagem (OE2), coerente com a implementação, como mostram o teste de fronteiras entre módulos e as restrições do banco. A integração da pesagem (OE4) foi atingida em simulação, com 11 de 12 cenários verificados integralmente e 1.177 de 1.178 testes aprovados. Foram atingidos parcialmente a implementação do reconhecimento (OE3), testada só com imagens sintéticas (105 a 170 ms por imagem em CPU), e a avaliação do protótipo (OE5), cujo protocolo não foi executado na parte do reconhecimento. Nenhum objetivo deixou de ser atendido, mas os parcialmente atingidos restringem as conclusões à lógica e à integração do sistema.

A contribuição está em reunir essas funções em código aberto, reproduzível sem equipamentos e sob regras explícitas de falha segura, com intervenção rastreável e histórico auditável. Como limitações, não houve testes com balança, cancela ou câmera físicas; faltam o protocolo de um indicador real e a cancela serial; o acerto e a latência do reconhecimento com imagens reais não foram medidos; os parâmetros de demonstração não foram calibrados; a adequação à LGPD não foi avaliada; e o protótipo considera uma única balança.

Como trabalhos futuros, derivados dessas limitações, propõe-se avaliar o reconhecimento com um acervo real autorizado ou com bases públicas brasileiras, medindo taxas por condição, latência e erros por caractere e por placa; integrar o protocolo de um indicador e a cancela serial; definir as câmeras e o critério de posição; calibrar os parâmetros; e testar o sistema com equipamentos, medindo eventuais ganhos de tempo e confiabilidade.

7. Referências

[Agrawal and Joshi 2022] Agrawal, S. and Joshi, K. D. (2022). Indian commercial truck license plate detection and recognition for weighbridge automation. arXiv:2211.13194.

[Avery Weigh-Tronix 2026] Avery Weigh-Tronix (2026). SmarTower unmanned weighbridge system. `https://www.averyweigh-tronix.com/en/product/smartower-unmanned-weighbridge-system/`. Acesso em: 08 nov. 2026.

[Axelson 2007] Axelson, J. (2007). *Serial Port Complete: COM Ports, USB Virtual COM Ports, and Ports for Embedded Systems*. Lakeview Research, 2 edition.

[Canny 1986] Canny, J. (1986). A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6):679–698.

[Chang et al. 2004] Chang, S.-L., Chen, L.-S., Chung, Y.-C., and Chen, S.-W. (2004). Automatic license plate recognition. *IEEE Transactions on Intelligent Transportation Systems*, 5(1):42–53.

[CONTRAN 2022] CONTRAN (2022). Resolução CONTRAN nº 969, de 20 de junho de 2022: Placas de Identificação de Veículos (PIV). Diário Oficial da União, Seção 1, p. 57, 24 jun. 2022.

[Docker 2026] Docker (2026). Docker documentation. `https://docs.docker.com/`. Acesso em: 08 nov. 2026.

[Du et al. 2013] Du, S., Ibrahim, M., Shehata, M., and Badawy, W. (2013). Automatic license plate recognition (ALPR): A state-of-the-art review. *IEEE Transactions on Circuits and Systems for Video Technology*, 23(2):311–325.

[ESSMANN 2026] ESSMANN (2026). ESSMANN license plate recognition. `https://essmann.com/en/?p=48991`. Acesso em: 08 nov. 2026.

[Gomes 2016] Gomes, B. M. P. (2016). Automação de balança rodoviária. Monografia (Especialização em Automação Industrial), Universidade Tecnológica Federal do Paraná, Curitiba.

[Gonzalez and Woods 2008] Gonzalez, R. C. and Woods, R. E. (2008). *Digital Image Processing*. Prentice Hall, 3 edition.

[Gracia et al. 2025] Gracia, M. D., Mar-Ortiz, J., and Vargas, M. (2025). Truck appointment scheduling: A review of models and algorithms. *Mathematics*, 13(3):503.

[Jaided AI 2025] Jaided AI (2025). EasyOCR documentation. `https://www.jaided.ai/easyocr/`. Acesso em: 01 jun. 2026.

[Laroca et al. 2022] Laroca, R., Cardoso, E. V., Lucio, D. R., Estevam, V., and Menotti, D. (2022). On the cross-dataset generalization in license plate recognition. In *International Conference on Computer Vision Theory and Applications (VISAPP)*, pages 166–178.

[Laroca et al. 2018] Laroca, R., Severo, E., Zanlorensi, L. A., Oliveira, L. S., Gonçalves, G. R., Schwartz, W. R., and Menotti, D. (2018). A robust real-time automatic license plate recognition based on the YOLO detector. In *International Joint Conference on Neural Networks (IJCNN)*, pages 1–10.

[Laroca et al. 2021] Laroca, R., Zanlorensi, L. A., Gonçalves, G. R., Todt, E., Schwartz, W. R., and Menotti, D. (2021). An efficient and layout-independent automatic license plate recognition system based on the YOLO detector. *IET Intelligent Transport Systems*, 15(4):483–503.

[Nygard 2011] Nygard, M. (2011). Documenting architecture decisions. `https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions`. Acesso em: 08 nov. 2026.

[OpenCV 2025] OpenCV (2025). OpenCV documentation. `https://docs.opencv.org/`. Acesso em: 01 jun. 2026.

[OWASP Foundation 2026] OWASP Foundation (2026). Password storage cheat sheet. `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html`. Acesso em: 07 nov. 2026.

[PostgreSQL 2026] PostgreSQL (2026). PostgreSQL 18 documentation. `https://www.postgresql.org/docs/18/`. Acesso em: 08 nov. 2026.

[pySerial 2025] pySerial (2025). pySerial documentation. `https://pyserial.readthedocs.io/`. Acesso em: 01 jun. 2026.

[Ramírez 2025] Ramírez, S. (2025). FastAPI documentation. `https://fastapi.tiangolo.com/`. Acesso em: 01 jun. 2026.

[Rao et al. 2025] Rao, T., Rawat, S., Gupta, A., Mehra, N., Dhondiyal, S. A., and Kapil, D. (2025). Comprehensive study on automatic number plate recognition with Python using OpenCV and EasyOCR. In *Challenges in Information, Communication and Computing Technology*, pages 501–505. CRC Press.

[React 2026] React (2026). React documentation. `https://react.dev/`. Acesso em: 08 nov. 2026.

[Receita Federal do Brasil 2024] Receita Federal do Brasil (2024). Instrução normativa RFB nº 2.229, de 15 de outubro de 2024: CNPJ alfanumérico. Diário Oficial da União, Seção 1, p. 84, 16 out. 2024.

[TypeScript 2026] TypeScript (2026). TypeScript documentation. `https://www.typescriptlang.org/docs/`. Acesso em: 08 nov. 2026.

[Valle Júnior and Veronez Júnior 2018] Valle Júnior, S. and Veronez Júnior, A. (2018). Atualização tecnológica do processo de pesagem de veículos por meio de tecnologia RFID. In *22º Seminário de Automação e TI (ABM Week)*, pages 14–23, São Paulo.

[WHATWG 2026] WHATWG (2026). HTML Living Standard: Server-Sent Events. `https://html.spec.whatwg.org/multipage/server-sent-events.html`. Acesso em: 08 nov. 2026.